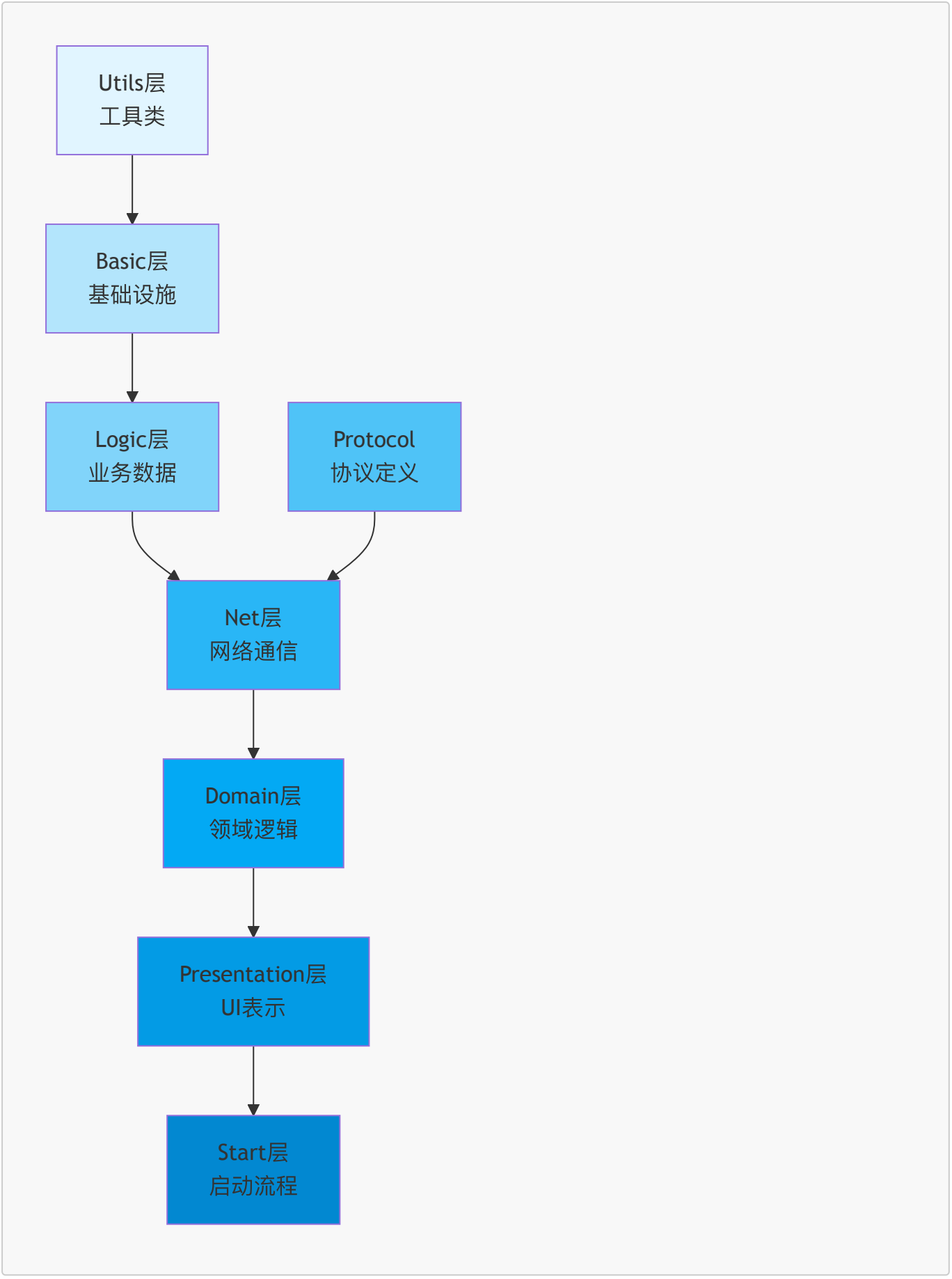
LOA客户端分层架构文档

一、架构总览

1.1 层次结构（自下而上）


Utils层
工具类
Basic层
基础设施
Logic层
业务数据
Protocol
协议定义
Net层
网络通信
Domain层
领域逻辑
Presentation层
UI表示
Start层
启动流程

1.2 与服务端对比

层次	服务端	客户端	说明
1	Utils	Utils	完全一致
2	Basic	Basic	完全一致
3	Logic	Logic	一致（原Data层）
4	Net（含Protocol）	Net（含Protocol）	**已对齐**
5	Domain	Domain	完全一致
6	-	Presentation	客户端特有（UI层）
7	Start	Start	完全一致

二、各层详细说明

2.1 Utils层（工具层）

位置： `Assets/Game/Scripts/Utils/`
程序集： `Utils.asmdef`

职责：

- 提供通用工具类
- 不依赖任何业务逻辑
- 可被所有上层使用

依赖：

- Framework（框架层）
- SuperScrollView（第三方UI组件）

关键类：

- `Utils.cs` - 日志、调试工具
- `InfiniWheel.cs` - UI滚轮组件

2.2 Basic层（基础设施层）

位置： `Assets/Game/Scripts/Basic/`
程序集： `Basic.asmdef`

职责：

- 事件系统（Event.cs, Monitor.cs）
- 流程管理（Flow.cs）
- 启动管理（Gate.cs）

依赖：

- Utils

- Framework

关键机制：

```
// 全局事件总线
Game.Event.Instance.Fire("EventName", data);
Game.Event.Instance.Add("EventName", callback);

// 属性监听（数据变更通知）
Data.Instance.after.Register(Data.Type.Home, OnHomeChanged);
```

跨层通信：

- 低层→高层：使用事件机制（Event/Monitor）
- 避免直接向上引用

2.3 Logic层（业务数据层）

位置： `Assets/Game/Scripts/Logic/`
程序集： `Logic.asmdef`

职责：

- 管理业务数据状态（Data.cs - 单例）
- 定义Logic数据模型（与Protocol解耦）
- 数据持久化（PBLocal.cs）

依赖：

- Utils
- Basic
- Framework
- LitJson
- Unity.TextMeshPro

关键数据模型： `Logic/Models/GameData.cs`

```
namespace Game
{
    public class MapData              // 地图块数据
    public class SceneData            // 场景数据
    public class HomeData             // 主页数据
    public class CharacterData        // 角色数据
    public class InitializeData       // 初始化数据
    public class OptionData           // 选项面板数据
    public class OptionItemData       // 选项项数据
    public class WorldMapData         // 世界地图数据
    public class UILockData           // UI锁定数据
    public class TutorialData         // 教程数据
```

```
    public class DialogueData          // 对话数据
    public class InformationData       // 消息数据
}
```

字段命名规则：

- **使用camelCase**（与Protocol保持一致）
- 示例：`pos`, `name`, `scene`, `characters`

核心类： `Data.cs`

```
public class Data : Singleton<Data>
{
    // 数据访问
    public HomeData Home { get; set; }
    public InitializeData Initialize { get; set; }
    public List<MapData> Maps { get; set; }
    public List<CharacterData> Characters { get; set; }

    // 数据变更通知
    public Monitor after = new Monitor();
    public void Change(Type type, object value);
}
```

2.4 Net层（网络通信层）

位置： `Assets/Game/Scripts/Network/`
程序集： `Net.asmdef`

职责：

- 网络通信（Net.cs）
- **Protocol协议定义**（Protocol/Protocol.cs）
- Protocol→Logic转换（ProtocolConverter.cs）

依赖：

- Protocol（子程序集，位于Network/Protocol/）
- Logic
- Basic
- Framework
- Utils

子程序集：Protocol

- **位置：** `Assets/Game/Scripts/Network/Protocol/`
- **程序集：** `Protocol.asmdef`
- **命名空间：** `Game.Protocol`

Protocol定义示例：

```
namespace Game.Protocol
{
    public class Map : Base          // 地图协议
    public class Scene : Base        // 场景协议
    public class Home : Base         // 主页协议
    public class Initialize : Base   // 初始化协议
    public class Option : Base       // 选项协议
    // ... 更多Protocol类
}
```

转换器： `ProtocolConverter.cs`

```
namespace Game
{
    public static class ProtocolConverter
    {
        // 扩展方法：Protocol → Logic
        public static MapData ToLogic(this Protocol.Map protocol);
        public static HomeData ToLogic(this Protocol.Home protocol);
        // ... 更多转换方法
    }
}
```

数据流向：

```
服务器 → Protocol (Net层) → ProtocolConverter → Logic数据模型 → 应用使用
```

核心类： `Net.cs`

```
public class Net : Singleton<Net>
{
    // 发送Protocol消息
    public void Send(Protocol.Base message);

    // 接收并处理Protocol消息
    private void Decode(Protocol.Base message);
}
```

Protocol.Processed()方法：

- 在Net层执行
- 调用ProtocolConverter转换为Logic数据
- 更新Data.Instance

- 触发事件通知上层

2.5 Domain层（领域层）

位置： `Assets/Game/Scripts/Domain/`
程序集： `Domain.asmdef`

职责：

- 领域业务逻辑
- Protocol处理器（ProtocolHandler.cs - 占位符）

依赖：

- Net
- Logic
- Utils
- Basic
- Framework

当前状态：

- 基础框架已就绪
- ProtocolHandler为占位符（TODO：从Protocol.cs迁移业务逻辑）

2.6 Presentation层（UI表示层）

位置： `Assets/Game/Scripts/UI/`
程序集： `Presentation.asmdef`

职责：

- 所有UI界面和交互
- UI动画
- 用户输入处理

依赖：

- Domain
- Logic
- Utils
- Basic
- Framework
- Net
- LitJson
- DOTweenModules（UI动画扩展）
- SuperScrollView
- Unity.TextMeshPro
- Unity.Addressables

- UnityEngine.Purchasing

precompiled DLL：

- DOTween.dll
- Newtonsoft.Json.dll
- Google.Protobuf.dll

关键UI组件：

- `UI.cs` - UI管理器
- `Home.cs` - 主页界面
- `Start.cs` - 启动界面
- `Initialize.cs` - 初始化界面
- `Option.cs` - 选项面板
- `Tutorial.cs` - 教程系统
- `Story.cs` - 剧情对话

数据访问规则：

- ✓ 使用Logic数据模型（MapData, HomeData等）
- ✗ 不直接访问Protocol类型（除DataPair, Chat等特殊情况）

2.7 Start层（启动层）

位置： `Assets/Game/Scripts/Start/`
程序集： `Start.asmdef`

职责：

- 游戏启动流程编排
- 依赖注入和初始化

依赖：

- Framework
- Basic
- Logic
- Net
- Domain
- Presentation

关键类：

- `Gate.cs` - 启动流程管理

三、关键技术点

3.1 数据类型系统

Protocol类型（网络层专用）

```
namespace Game.Protocol
{
    public class Home : Base
    {
        public Scene scene;
        public Characters characters;
        public Dictionary<string, Resource> resouse;
        public List<int[]> area;
        // ... Protocol字段（camelCase，Protobuf生成）
    }
}
```

Logic数据模型（应用层通用）

```
namespace Game
{
    public class HomeData
    {
        public SceneData scene { get; set; }
        public List<CharacterData> characters { get; set; }
        public Dictionary<string, ResourceInfo> resouse { get; set; }
        public List<int[]> area { get; set; }
        // ... Logic字段（camelCase，手动定义）
    }
}
```

为什么要分离？

- **解耦**：应用逻辑不依赖网络协议
- **灵活**：Protocol变更不影响应用层
- **清晰**：职责明确，Protocol仅用于序列化/反序列化

3.2 跨层通信机制

低层→高层：事件机制

```
// Basic层触发事件
Game.Event.Instance.Fire("UI.Event.Click", buttonData);

// Presentation层监听事件
Game.Event.Instance.Add("UI.Event.Click", OnButtonClick);
```

数据变更通知

```
// Logic层触发
Data.Instance.after.Fire(Data.Type.Home, newHomeData);

// Presentation层监听
Data.Instance.after.Register(Data.Type.Home, OnHomeChanged);
```

3.3 命名规范

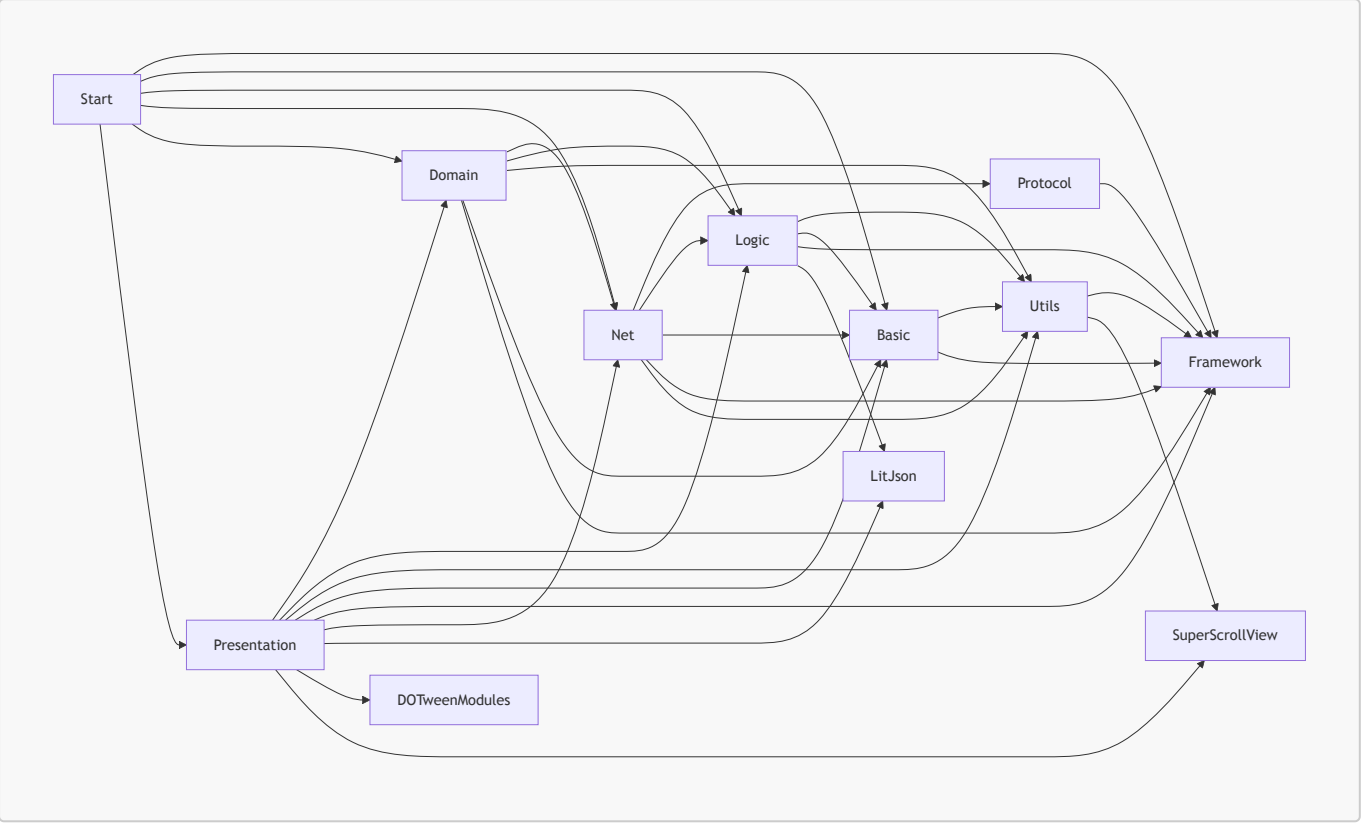
类型	命名规则	示例
Protocol类	PascalCase类名	`Protocol.Home`, `Protocol.Map`
Protocol字段	camelCase	`scene`, `characters`, `pos`
Logic数据模型	PascalCase类名 + Data后缀	`HomeData`, `MapData`
Logic字段	camelCase	`scene`, `characters`, `pos`
UI组件	PascalCase	`Home`, `Start`, `OptionButton`

特殊情况：

- `OptionItemData` - 数据模型（避免与UI组件`OptionItem`冲突）

四、程序集依赖配置

4.1 依赖关系图



4.2 各层.asmdef配置

Utils.asmdef

```
{
    "name": "Utils",
    "references": ["Framework", "SuperScrollView"]
}
```

Basic.asmdef

```
{
    "name": "Basic",
    "references": ["Utils", "Framework"]
}
```

Logic.asmdef

```
{
    "name": "Logic",
    "references": ["Utils", "Basic", "Framework", "LitJson",
"Unity.TextMeshPro"],
    "precompiledReferences": ["Google.Protobuf.dll", "Newtonsoft.Json.dll"]
}
```

Protocol.asmdef

```
{
    "name": "Protocol",
    "references": ["Basic", "Framework"],
    "precompiledReferences": ["Google.Protobuf.dll"]
}
```

Net.asmdef

```
{
    "name": "Net",
    "references": ["Protocol", "Logic", "Basic", "Framework", "Utils"],
    "precompiledReferences": ["Google.Protobuf.dll", "Newtonsoft.Json.dll"]
}
```

Domain.asmdef

```
{
    "name": "Domain",
    "references": ["Net", "Logic", "Utils", "Basic", "Framework"]
}
```

Presentation.asmdef

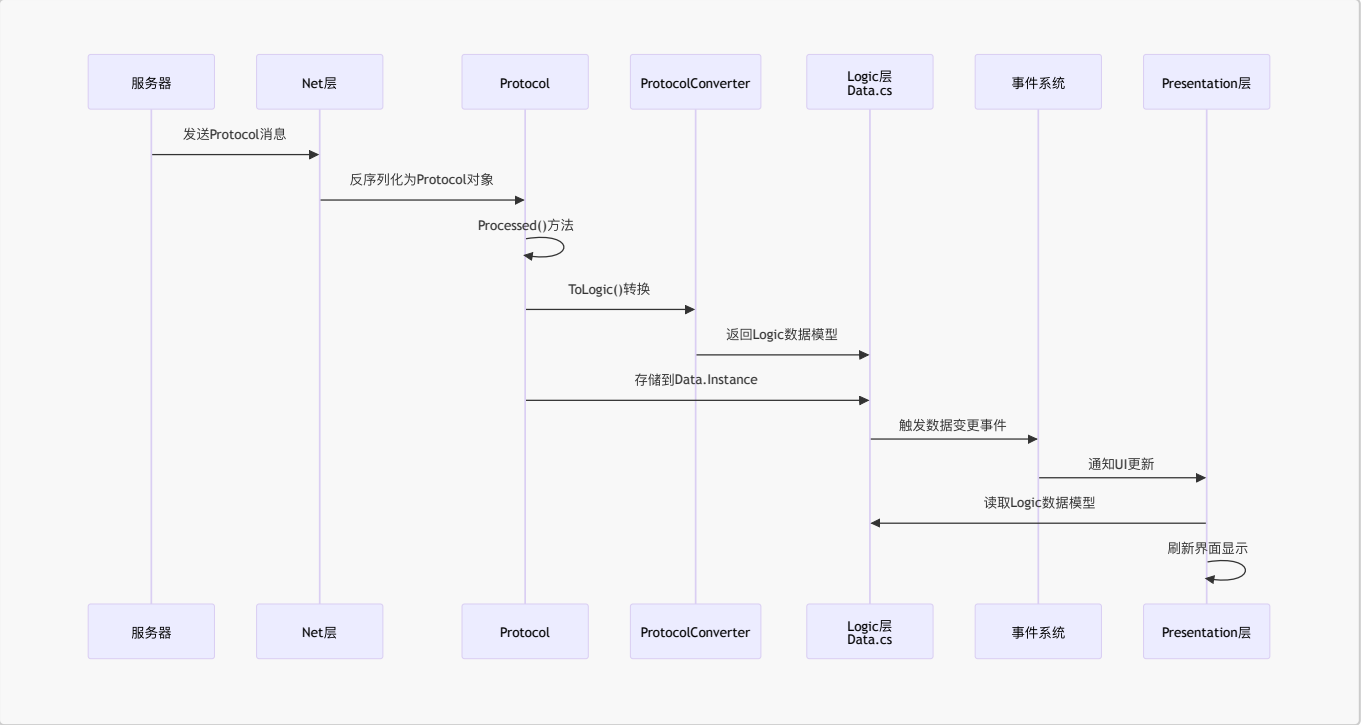
```
{
    "name": "Presentation",
    "references": [
        "Domain", "Logic", "Utils", "Basic", "Framework", "Net",
        "LitJson", "DOTweenModules", "SuperScrollView",
        "Unity.TextMeshPro", "Unity.Addressables", "UnityEngine.Purchasing"
    ],
    "precompiledReferences": [
        "DOTween.dll", "Newtonsoft.Json.dll", "Google.Protobuf.dll"
    ]
}
```

Start.asmdef

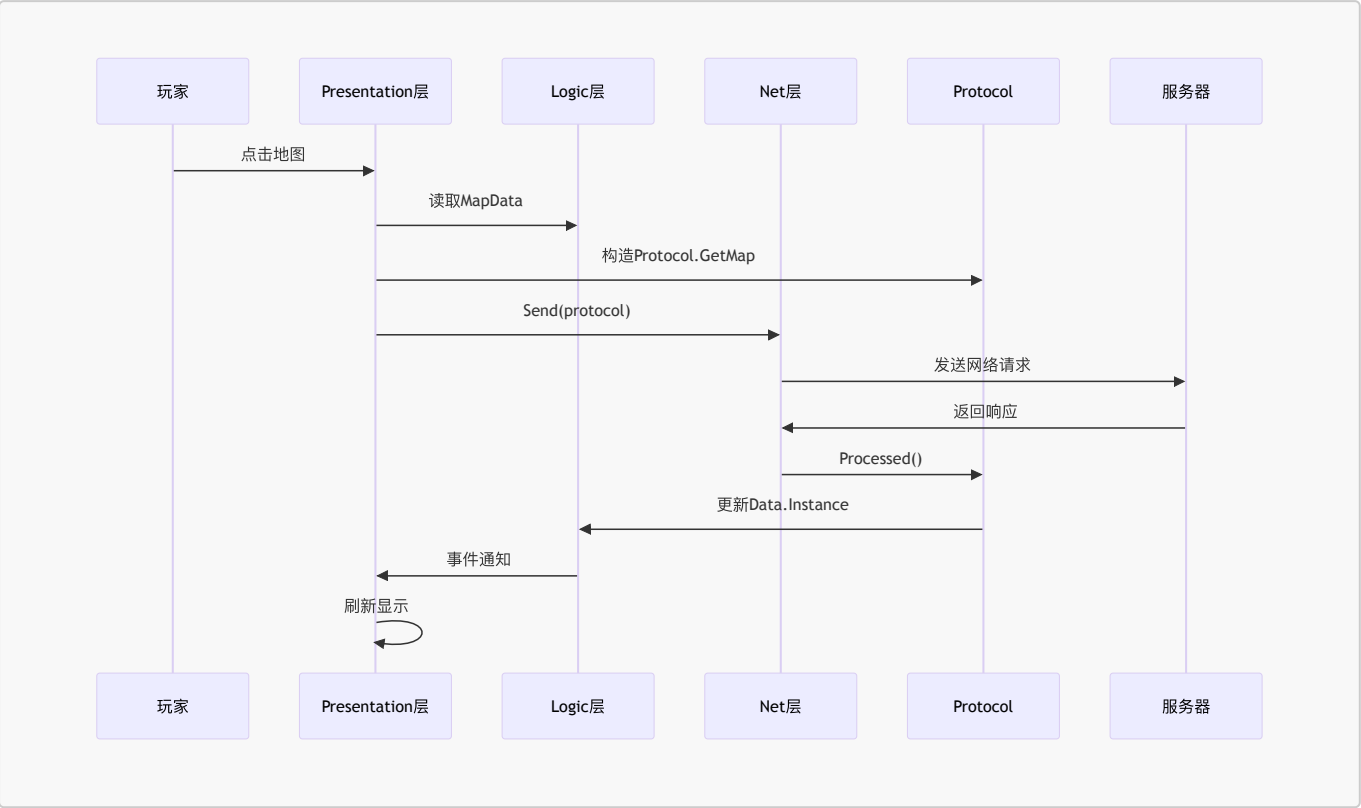
```
{
    "name": "Start",
    "references": ["Framework", "Basic", "Logic", "Net", "Domain",
"Presentation"]
}
```

五、数据流示例

5.1 网络消息接收流程



5.2 用户操作流程



六、关键文件索引

6.1 架构核心文件

文件路径	说明
`Logic/Data.cs`	业务数据管理器（单例）

文件路径	说明
`Logic/Models/GameData.cs`	Logic数据模型定义
`Network/Net.cs`	网络通信管理器
`Network/Protocol/Protocol.cs`	Protocol协议定义（约3000行）
`Network/ProtocolConverter.cs`	Protocol→Logic转换器
`Basic/Event.cs`	全局事件总线
`Basic/Monitor.cs`	属性监听器
`UI/UI.cs`	UI管理器

6.2 程序集定义文件

asmdef文件	层次
`Utils/Utils.asmdef`	Utils层
`Basic/Basic.asmdef`	Basic层
`Logic/Logic.asmdef`	Logic层
`Network/Protocol/Protocol.asmdef`	Protocol子程序集
`Network/Net.asmdef`	Net层
`Domain/Domain.asmdef`	Domain层
`UI/Presentation.asmdef`	Presentation层
`Start/Start.asmdef`	Start层

6.3 第三方库程序集

asmdef文件	说明
`ThirdParty/Json/LitJson.asmdef`	LitJson源代码程序集
`ThirdParty/Demigiant/DOTween/Modules/DOTweenModules.asmdef`	DOTween扩展方法
`ThirdParty/SuperScrollView/Scripts/SuperScrollView.asmdef`	滚动视图组件

七、架构改造历史

改造前（v1.0）

- **架构模式**：扁平化管理器协作
- **Protocol位置**：独立层
- **Data层**：直接使用Protocol类型
- **程序集**：无.asmdef，所有代码在默认程序集

改造后（v2.0）

- **架构模式**：严格分层架构（对齐服务端）
- **Protocol位置**：归属Net层
- **Logic层**：独立数据模型，与Protocol解耦
- **程序集**：每层独立.asmdef，依赖显式声明

主要变更（SVN版本149）

1. Protocol从独立层→Net层
2. Data层→Logic层
3. 创建11个Logic数据模型
4. 创建ProtocolConverter转换器
5. UI层60+文件类型迁移
6. 8个.asmdef配置
7. 3个第三方库程序集

八、开发规范

8.1 添加新Protocol

1. 在`Network/Protocol/Protocol.cs`中定义Protocol类
2. 在`Logic/Models/GameData.cs`中定义对应的Logic数据模型
3. 在`Network/ProtocolConverter.cs`中添加转换方法
4. 在Protocol的`Processed()`中调用转换器
5. 在`Logic/Data.cs`中添加属性和Type枚举

8.2 添加新UI界面

1. 继承`UI.Core`基类
2. 在`Presentation.asmdef`的命名空间中
3. 只访问Logic数据模型，不直接用Protocol
4. 使用事件机制与低层通信

8.3 跨层调用规则

✓ 允许：

- 高层→低层：直接调用
- 低层→高层：通过事件机制

✕ 禁止：

- 低层直接调用高层
- 跨程序集访问未在references中声明的类型

九、常见问题

Q1: Logic数据模型和Protocol的区别?

A:

- Protocol：网络传输格式，由Protobuf生成，字段不可控
- Logic数据模型：应用内部数据，手动定义，可扩展

Q2: 为什么字段用camelCase而非C#标准的PascalCase?

A:

- 保持与Protocol字段命名一致
- 减少转换时的命名转换成本
- UI层代码可以平滑过渡

Q3: UI层什么时候需要Protocol?

A:

- 仅在需要发送网络消息时：`Net.Instance.Send(new Protocol.XXX())`
- 或处理特殊Protocol类型（DataPair, Chat等）
- 其他情况都应使用Logic数据模型

Q4: 如何确保依赖不违反分层?

A:

- .asmdef的`references`强制约束
- 编译器会报错如果违反依赖规则
- 例如：Logic层无法引用Net层（references中没有）

十、后续优化建议

10.1 待完成功能

- ☐ ProtocolHandler完整实现（从Protocol.cs迁移业务逻辑）
- ☐ CodelessIAPButton恢复（添加UnityEngine.Purchasing.Codeless引用）
- ☐ PlayerState功能恢复

10.2 架构优化

- ☐ 考虑引入中介者模式减少UI与Logic的直接耦合
- ☐ Domain层扩充领域服务
- ☐ 完善单元测试覆盖

10.3 性能优化

- ☐ ProtocolConverter转换器性能测试
- ☐ 大量数据场景下的内存占用分析
- ☐ 事件系统性能评估

附录：架构对比表

维度	改造前	改造后
架构风格	扁平化协作	严格分层
Protocol位置	独立层	Net层子程序集
数据模型	直接用Protocol	独立Logic模型
程序集数量	1个（默认）	11个（含第三方）
依赖管理	无约束	.asmdef强约束
与服务端一致性	低	高
可维护性	中	高
类型安全	中	高

文档版本： 2.0
最后更新： 2026-02-17
维护者： 架构组